

UNIVERSIDAD ESTATAL AMAZÓNICA

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

TAREA: REGISTRO DE ESTUDIANTE USANDO ESTRUCTURAS DE DATOS EN C#

Estudiante: Milton Manuel Mosquera Quiñones

Carrera: Tecnologías de la Información

Nivel: Tercero

Usuario GitHub: Mil2003

Correo GitHub: <https://github.com/Mil2003/pruea>

Fecha: 12 de junio de 2025

INTRODUCCIÓN

En el desarrollo de aplicaciones de software, la gestión eficiente de datos es fundamental para crear sistemas robustos y escalables. Este documento presenta la implementación de una estructura de datos en C# para manejar el registro de estudiantes, utilizando conceptos fundamentales como clases, arrays y propiedades.

El objetivo principal es diseñar una clase `Estudiante` que permita almacenar información personal básica como identificación, nombres, apellidos, dirección y múltiples números telefónicos. Esta implementación demuestra el uso de arrays para manejar colecciones de datos homogéneos (teléfonos) y la encapsulación de datos mediante propiedades en C#.

La estructura propuesta no solo cumple con los requerimientos técnicos solicitados, sino que también establece una base sólida para futuros desarrollos que requieran manejo de información estudiantil en sistemas educativos.

DESARROLLO

Código Fuente en C#

A continuación se presenta la implementación completa de la clase `Estudiante` con todos los atributos requeridos:

```
using System;

namespace RegistroEstudiante
{
    /// <summary>
    /// Clase que representa un estudiante con sus datos personales
    /// Incluye manejo de múltiples teléfonos mediante arrays
    /// </summary>
    public class Estudiante
    {
```

```

// Campos privados para encapsular los datos
private int id;
private string nombres;
private string apellidos;
private string direccion;
private string[] telefonos; // Array para almacenar 3 teléfonos

/// <summary>
/// Constructor de la clase Estudiante
/// </summary>
/// <param name="id">Identificador único del estudiante</param>
/// <param name="nombres">Nombres del estudiante</param>
/// <param name="apellidos">Apellidos del estudiante</param>
/// <param name="direccion">Dirección de residencia</param>
public Estudiante(int id, string nombres, string apellidos, string
direccion)
{
    this.id = id;
    this.nombres = nombres ?? throw new
ArgumentNullException(nameof(nombres));
    this.apellidos = apellidos ?? throw new
ArgumentNullException(nameof(apellidos));
    this.direccion = direccion ?? throw new
ArgumentNullException(nameof(direccion));

    // Inicializar array de teléfonos con capacidad para 3 números
    this.telefonos = new string[3];

    // Inicializar con valores vacíos para evitar referencias nulas
    for (int i = 0; i < telefonos.Length; i++)
    {
        telefonos[i] = "";
    }
}

// Propiedades públicas para acceder a los datos (Get/Set)

/// <summary>
/// Identificador único del estudiante
/// </summary>
public int Id
{
    get { return id; }
    set
    {
        if (value > 0)
            id = value;
        else
            throw new ArgumentException("El ID debe ser mayor a 0");
    }
}

/// <summary>
/// Nombres del estudiante
/// </summary>
public string Nombres
{
    get { return nombres; }
    set
    {
        if (!string.IsNullOrEmpty(value))
            nombres = value.Trim();
        else

```

```

        throw new ArgumentException("Los nombres no pueden estar
vacíos");
    }
}

/// <summary>
/// Apellidos del estudiante
/// </summary>
public string Apellidos
{
    get { return apellidos; }
    set
    {
        if (!string.IsNullOrEmpty(value))
            apellidos = value.Trim();
        else
            throw new ArgumentException("Los apellidos no pueden estar
vacíos");
    }
}

/// <summary>
/// Dirección de residencia del estudiante
/// </summary>
public string Direccion
{
    get { return direccion; }
    set
    {
        if (!string.IsNullOrEmpty(value))
            direccion = value.Trim();
        else
            throw new ArgumentException("La dirección no puede estar
vacía");
    }
}

/// <summary>
/// Array de teléfonos del estudiante (máximo 3)
/// </summary>
public string[] Telefonos
{
    get { return telefonos; }
}

// Métodos específicos para manejar los teléfonos

/// <summary>
/// Establece un teléfono en la posición especificada
/// </summary>
/// <param name="indice">Posición en el array (0, 1 o 2)</param>
/// <param name="telefono">Número telefónico</param>
public void SetTelefono(int indice, string telefono)
{
    if (indice >= 0 && indice < telefonos.Length)
    {
        telefonos[indice] = telefono ?? "";
    }
    else
    {
        throw new IndexOutOfRangeException("Índice de teléfono debe
estar entre 0 y 2");
    }
}

```

```

    /// <summary>
    /// Obtiene un teléfono de la posición especificada
    /// </summary>
    /// <param name="indice">Posición en el array (0, 1 o 2)</param>
    /// <returns>Número telefónico en la posición especificada</returns>
    public string GetTelefono(int indice)
    {
        if (indice >= 0 && indice < telefonos.Length)
        {
            return telefonos[indice];
        }
        else
        {
            throw new IndexOutOfRangeException("Índice de teléfono debe
estar entre 0 y 2");
        }
    }

    /// <summary>
    /// Método para mostrar toda la información del estudiante
    /// </summary>
    /// <returns>Cadena con toda la información formateada</returns>
    public override string ToString()
    {
        string info = $"ID: {id}\n";
        info += $"Nombres: {nombres}\n";
        info += $"Apellidos: {apellidos}\n";
        info += $"Dirección: {direccion}\n";
        info += "Teléfonos:\n";

        for (int i = 0; i < telefonos.Length; i++)
        {
            info += $"  Teléfono {i + 1}:
{(string.IsNullOrEmpty(telefonos[i]) ? "No registrado" : telefonos[i])}\n";
        }

        return info;
    }

    /// <summary>
    /// Método para validar si todos los datos obligatorios están completos
    /// </summary>
    /// <returns>True si los datos están completos, False en caso
contrario</returns>
    public bool DatosCompletos()
    {
        return id > 0 &&
            !string.IsNullOrEmpty(nombres) &&
            !string.IsNullOrEmpty(apellidos) &&
            !string.IsNullOrEmpty(direccion);
    }
}

/// <summary>
/// Clase principal para demostrar el uso de la clase Estudiante
/// </summary>
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("=== SISTEMA DE REGISTRO DE ESTUDIANTES ===\n");

        try
    
```

```

{
    // Crear una instancia de estudiante
    Estudiante estudiante1 = new Estudiante(
        1001,
        "Milton Manuel",
        "Mosquera Quiñones",
        "Esmeraldas, Ecuador"
    );

    // Asignar teléfonos usando el método SetTelefono
    estudiante1.SetTelefono(0, "0987654321");
    estudiante1.SetTelefono(1, "062345678");
    estudiante1.SetTelefono(2, "0991234567");

    // Mostrar información del estudiante
    Console.WriteLine("Información del Estudiante:");
    Console.WriteLine(estudiante1.ToString());

    // Verificar si los datos están completos
    Console.WriteLine($"Datos completos:
{(estudiante1.DatosCompleto() ? "Sí" : "No")}");

    // Demostrar acceso individual a teléfonos
    Console.WriteLine("\nAcceso individual a teléfonos:");
    for (int i = 0; i < 3; i++)
    {
        Console.WriteLine($"Teléfono {i + 1}:
{estudiante1.GetTelefono(i)}");
    }

    // Crear un array de estudiantes para demostrar manejo múltiple
    Console.WriteLine("\n=== REGISTRO MÚLTIPLE DE ESTUDIANTES ===");

    Estudiante[] listaEstudiantes = new Estudiante[3];

    // Llenar el array con estudiantes de ejemplo
    listaEstudiantes[0] = estudiante1;

    listaEstudiantes[1] = new Estudiante(1002, "Ana María", "García
López", "Quito, Ecuador");
    listaEstudiantes[1].SetTelefono(0, "0998877665");

    listaEstudiantes[2] = new Estudiante(1003, "Carlos Eduardo",
"Ramírez Silva", "Guayaquil, Ecuador");
    listaEstudiantes[2].SetTelefono(0, "0987766554");
    listaEstudiantes[2].SetTelefono(1, "042234567");

    // Mostrar todos los estudiantes registrados
    for (int i = 0; i < listaEstudiantes.Length; i++)
    {
        Console.WriteLine($"--- Estudiante {i + 1} ---");
        Console.WriteLine(listaEstudiantes[i].ToString());
    }
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}

Console.WriteLine("\nPresione cualquier tecla para salir...");
Console.ReadKey();
}
}
}

```

Enlace Público GitHub

Repositorio: https://github.com/Mil2003/pruea/tree/main/UEA/tercer_semestre_ed

Características Técnicas Implementadas

1. **Encapsulación:** Los campos son privados y se acceden mediante propiedades públicas.
 2. **Validación de Datos:** Cada propiedad incluye validaciones para asegurar la integridad de los datos.
 3. **Manejo de Arrays:** Los teléfonos se almacenan en un array de tamaño fijo (3 elementos).
 4. **Métodos Especializados:** Implementación de métodos específicos para manejar los teléfonos individualmente.
 5. **Manejo de Excepciones:** Control de errores mediante try-catch y validaciones.
-

CONCLUSIÓN

La implementación de la clase `Estudiante` en C# demuestra la aplicación práctica de conceptos fundamentales de programación orientada a objetos y estructuras de datos. A través de este ejercicio, he logrado:

Aspectos Técnicos Logrados:

- Diseñar una clase robusta con encapsulación adecuada de datos
- Implementar el uso de arrays para manejar colecciones de datos homogéneos
- Aplicar validaciones de entrada para garantizar la integridad de los datos
- Crear métodos especializados para operaciones específicas con arrays

Aprendizajes Obtenidos: El desarrollo de esta tarea me ha permitido profundizar en el manejo de arrays en C# y comprender mejor la importancia de la validación de datos en aplicaciones reales. La implementación de propiedades con validación asegura que los datos del estudiante mantengan su integridad a lo largo del ciclo de vida del objeto.

Aplicabilidad Práctica: Esta estructura de datos puede ser fácilmente escalada para sistemas más complejos, como sistemas de gestión estudiantil completos. La base sólida establecida permite futuras expansiones como persistencia de datos, interfaces gráficas de usuario, y conexión con bases de datos.

El uso de arrays para los teléfonos demuestra ser una solución eficiente para casos donde se conoce el número máximo de elementos a almacenar, proporcionando acceso rápido y uso eficiente de memoria.

BIBLIOGRAFÍA

Albahari, J., & Albahari, B. (2022). *C# 10 in a Nutshell: The Definitive Reference* (1ª ed.). O'Reilly Media.

Deitel, P., & Deitel, H. (2020). *C# How to Program* (6ª ed.). Pearson Education.

Microsoft Corporation. (2024). *Documentación de C# - Guía de programación*. Microsoft Docs.
<https://docs.microsoft.com/es-es/dotnet/csharp/>

Skeet, J. (2019). *C# in Depth* (4ª ed.). Manning Publications.

Troelsen, A., & Japikse, P. (2021). *Pro C# 9 with .NET 5: Foundational Principles and Practices in Programming* (10ª ed.). Apress.